EcoScan - Diagramme de Classes

Description des 10 classes du diagramme UML - implementation reelle

Ce document reproduit fidelement le diagramme de classes de l'application EcoScan avec les attributs et methodes reels tels qu'implantes dans le code Flutter/Dart. Chaque classe UML est mise en correspondance avec le(s) fichier(s) source qui l'implementent.

1. Categorie -> implemente via ScanEntry._categoryData

Implemente dans : Frontend/lib/models/scan_entry.dart (ligne 28)

Attributs :

Attribut	Type	Valeurs / Description
id (category)	String	Identifiant de categorie : 'cardboard', 'plastique', 'verre', 'metal', 'organique', 'paper', 'electronique', 'trash', 'unknown', 'chaussures', 'vetement'
nom (name)	String	'Carton recyclable', 'Plastique recyclable', 'Verre recyclable', 'Metal recyclable', 'Dechet organique', 'Papier recyclable', 'Appareil DEEE', 'Dechet non recyclable', 'Objet non reconnu', 'Chaussures usagees', 'Vetement / Textile'
ConsigneTri (binColor)	String	Couleur de poubelle recommandee : 'Bleu', 'Jaune', 'Vert', 'Marron', 'Noir', 'Eco-point'
recyclable	bool	true / false - indique si le dechet est recyclable
dangerous	bool	true pour DEEE (electronique), false pour les autres
points	int	Points gagnes par scan : de 0 (unknown) a 40 (electronique)
co2Saved	double	CO2 evite en grammes : de 0.0 (trash) a 80.0 (metal)
recyclability	double	Taux de recyclabilite : 0.0 a 1.0 (verre = 1.0, metal = 0.97)

Methodes :

Methode	Description
afficherConsigne() -> ScanEntry.fromClassification()	fromClassification(String label, double confidence) -> ScanEntry Retourne un ScanEntry complet avec binColor (consigne de tri) calcule depuis le label YOLOv8
toJson() -> Map<String, dynamic>	Serialise le ScanEntry en JSON pour stockage SharedPreferences et Firestore
fromJson() -> factory ScanEntry	factory ScanEntry.fromJson(Map<String, dynamic> json) -> ScanEntry Deserialisition depuis JSON (SharedPreferences ou Firestore)

2. CentreTri -> Donnees statiques dans la carte

Implemente dans : Frontend/lib/screens/map/map_screen.dart

Attributs :

Attribut	Type	Description
id	String	Identifiant unique du centre de tri
nom	String	Ex : 'Geocycle Casablanca', 'Eco Zalagh Sidi Hajaj', 'SMRR Tit Mellil'
adresse	String	Adresse complete du centre
latitude	double	Coordonnee GPS latitude

longitude	double	Coordonnee GPS longitude (note : ecrit longtitude dans le diagramme)
categories	List<String>	Types de dechets acceptes : 'Plastique', 'Verre', 'Papier', 'Compost', 'Piles', 'Electronique', 'Textile', 'Huiles', 'Metal'

Methodes :

Methode	Description
afficherlocalisation()	Affiche le centre sur la carte interactive (flutter_map + OpenStreetMap) avec un marqueur cliquable
Filtrage par categorie	Permet de filtrer les centres affiches sur la carte par type de dechet accepte (9 filtres disponibles)

3. EcoBot -> ChatService + ChatNotifier

Implemente dans : Frontend/lib/services/chat_service.dart | Frontend/lib/providers/chat_provider.dart

Attributs :

Attribut	Type	Description
id (_dio)	Dio	Client HTTP configure vers le backend Gemini RAG (ecoscan-chatbot.onrender.com)
BaseConnaissance (_offlineFallback)	String (fonction)	Base de connaissances hors-ligne integree : reponses sur le tri des dechets, la navigation dans l'app et les contacts operateurs (FR / EN / AR / ES / Darija)
history	List<ChatMessage>	Historique des messages de la session en cours (envoye au backend pour contexte)
baseUrl	String	URL du backend depuis ApiConstants (ecoscan-chatbot.onrender.com)

Methodes :

Methode	Signature / Description
repondreQuestion() -> sendMessage()	sendMessage(String message, {List<Map> history = const []}) -> Future<String> Envoi du message au backend Gemini, retour nettoye (sans markdown **)
fournirConseil() -> _offlineFallback()	_offlineFallback(String message) -> String Repond localement si le backend est indisponible : 20+ categories (dechets, navigation, contacts)
isHealthy()	Future<bool> - Ping /health pour verifier la disponibilite du serveur backend
_cleanMarkdown()	Supprime les symboles markdown generes par Gemini : **, __, ##, *
_containsAny()	_containsAny(String text, List<String> keywords) -> bool - Detection de mots-cles
reset() (ChatNotifier)	Vide l'historique de conversation et reinitialise l'etat du chat

4. Utilisateur -> AppUser + UserModel

Implemente dans : Frontend/lib/providers/auth_provider.dart | Frontend/lib/models/user_model.dart

Attributs :

Attribut UML	Type	Attribut Dart / Description
id	String	uid - Identifiant unique Firebase Auth
nom	String	displayName - Prenom + Nom de l'utilisateur
email	String	email - Adresse email (Firebase Auth)
MotDePasse	String	Gere par Firebase Auth - non stocke en clair dans l'app
-	String	city - Ville de l'utilisateur (stockee dans Firestore)
-	String?	photoUrl - URL photo de profil (nullable)
-	int	points - Points eco cumules
-	int	scans - Nombre total de scans effectues
-	int	badges - Nombre de badges debloques
-	int	rank - Classement dans le leaderboard communautaire
-	int	level / xp / xpMax - Niveau et points d'experience (xpMax defaut : 200)
-	double	co2Saved - CO2 total evite en grammes
-	int	streak - Nombre de jours consecutifs d'utilisation
-	double	weeklyGoalKg / weeklyProgress - Objectif et avancement hebdomadaire
-	List<Map>	recentScans - Derniers scans de l'utilisateur

Methodes :

Methode UML	Methode Dart / Description
sAuthentifier()	login(String email, String password) -> Future<void> | Google signIn() Authentification Firebase + chargement du profil depuis Firestore
scannerDechet()	ScanApiService.classify(File imageFile) -> Future<ScanApiResult?> Appel API YOLOv8 cloud pour analyser l'image du dechet
consulterHistorique()	ScanHistoryNotifier.loadHistory() -> Future<void> Charge l'historique depuis SharedPreferences puis Firestore si vide
consulterStatistiques()	Stream userProvider + communityStatsProvider Flux temps reel : points, scans, co2, streak, level, classement
dialoguerAvecEcoBot()	ChatService.sendMessage(String message, {List history}) -> Future<String>
register()	register({firstName, lastName, email, password, city}) -> Future<void> Creation compte Firebase + document Firestore + email de verification
signOut()	Efface SharedPreferences + deconnexion Google + Firebase Auth
copyWith()	UserModel.copyWith(...) -> UserModel - Mise a jour partielle du modele utilisateur
AppUser.fromFirebase()	factory AppUser.fromFirebase(User fbUser, {String city}) -> AppUser Convertit un utilisateur Firebase en AppUser

5. ResultatScan -> ScanEntry

Implemente dans : Frontend/lib/models/scan_entry.dart

Attributs :

Attribut UML	Type	Attribut Dart / Description
id	String	id - Timestamp en millisecondes (identifiant unique du scan)
scoreconfiance	double	recyclability - Score de recyclabilite 0.0 -> 1.0 (confiance YOLOv8 x taux de la categorie)
consigneTri	String	binColor - Couleur de poubelle recommandee : 'Bleu', 'Jaune', 'Vert', 'Marron', 'Noir', 'Eco-point'
-	String	name - Nom lisible : 'Plastique recyclable', 'Verre recyclable'...
-	String	category - Label interne : 'plastique', 'verre', 'metal'...
-	bool	recyclable - true si le dechet est recyclable
-	bool	dangerous - true pour les DEEE (appareils electroniques)
-	int	points - Points gagnes (ajuste par confidence : pts * conf, clamp a pts/2..pts)
-	double	co2Saved - CO2 evite en grammes (ajuste par confidence)
-	DateTime	scannedAt - Horodatage du scan
-	String	mode - 'cloud' (YOLOv8 API) ou 'local' (TFLite embarque)

Methodes :

Methode UML	Methode Dart / Description
afficherResultat()	ScanResultScreen - Affiche name, binColor, points, co2Saved, recyclability avec code couleur et icone de la categorie
fromClassification()	static fromClassification(String label, double confidence) -> ScanEntry Construit un ScanEntry complet depuis le label YOLOv8 et le score de confiance
toJson()	Map<String, dynamic> - Serialisation pour SharedPreferences et Firestore
fromJson()	factory ScanEntry.fromJson(Map<String, dynamic> json) -> ScanEntry Deserialisation depuis JSON

6. Scan -> ScanApiService + ClassifierService

Implemente dans : Frontend/lib/services/scan_api_service.dart | Frontend/lib/services/classifier_service_io.dart

Attributs :

Attribut UML	Type	Attribut Dart / Description
id (topLabel)	String	Label principal detecte par YOLOv8 (traduit en categorie EcoScan via _translateLabel)
DateScan	DateTime	scannedAt dans le ScanEntry resultant - date et heure du scan
-	double	topConfidence - Score de confiance YOLOv8 (seuil minimum client : 0.40)
-	List<Detection>	detections - Liste de toutes les detections dans l'image (label + confidence)
-	Dio (static)	_dio - Client HTTP vers ecoscan-model-online.onrender.com (timeout connect 30s, receive 60s pour Render cold start)

Methodes :

Methode UML	Methode Dart / Description
analyserImage()	static classify(File imageFile) -> Future<ScanApiResult?> Envoi de l'image au backend YOLOv8 via multipart/form-data Retourne null si serveur indisponible (fallback TFLite)
obtenirConsignes()	static _translateLabel(String label) -> String Traduction de 80+ labels YOLO/TACO vers les categories EcoScan (Glass bottle -> verre, Drink can -> metal, etc.)
isAvailable()	static Future<bool> - Ping /health pour reveiller le serveur Render (cold start) A appeler au demarrage de l'app
ClassifierService.classify()	Fallback TFLite local (EfficientNetV2B0) si le cloud YOLOv8 est indisponible

7. HistoriqueScan -> ScanHistoryNotifier

Implemente dans : Frontend/lib/providers/scan_history_provider.dart

Attributs :

Attribut UML	Type	Attribut Dart / Description
id	String	UID Firebase de l'utilisateur - cle Firestore users/{uid}/scans
listescans	List<ScanEntry>	state - Liste des entrees de scan (max 200 via Firestore stream, 50 charges initialement)
-	double (getter)	totalCo2Saved - Somme du CO2 evite de tous les scans
-	int (getter)	totalPoints - Somme des points gagnes sur tous les scans
-	int (getter)	recyclableCount - Nombre de scans avec un dechet recyclable

Methodes :

Methode UML	Methode Dart / Description
ajouterScan()	addScan(ScanEntry entry) -> void Ajoute le scan en tete de liste + persist en JSON dans SharedPreferences
ConsulterHistorique()	loadHistory() -> Future<void> 1. Charge depuis SharedPreferences (rapide, hors-ligne) 2. Si vide : charge depuis Firestore (users/{uid}/scans, limit 50)
_loadFromFirestore()	_loadFromFirestore(String uid) -> Future<void> Synchronisation Firestore -> etat local + sauvegarde SharedPreferences
_saveToPrefs()	Serialise l'etat complet (List<ScanEntry>) en JSON dans SharedPreferences
scanHistoryStreamProvider	StreamProvider<List<ScanEntry>> - Flux Firestore temps reel (limit 200) pour les ecrans de statistiques

8. Statistique -> CommunityStats + UserModel

Implemente dans : Frontend/lib/providers/community_stats_provider.dart

Attributs - Statistiques communautaires (CommunityStats) :

Attribut UML	Type	Attribut Dart / Calcul
id	Firestore	Document community_totals/stats - cle du document des totaux globaux
nombreScans	int	scans - Calcule : totalPoints / 10 (10 pts en moyenne par scan)
co2Evite	int	kgRecycled - Calcule : scans x 0.35 kg de CO2 evite
point	int	users - Nombre d'utilisateurs actifs via Firestore count()
-	double	progress - Progression objectif communautaire 50 000 pts (0.0 -> 1.0)

Attributs - Statistiques utilisateur (UserModel) :

Attribut UML	Type	Attribut Dart / Description
nombreScans	int	scans - Nombre total de scans de l'utilisateur
co2Evite	double	co2Saved - CO2 total evite en grammes
point	int	points - Points eco cumules
-	int	streak - Jours consecutifs d'utilisation
-	int	level / xp - Niveau et experience de l'utilisateur
-	double	weeklyProgress - Avancement objectif hebdomadaire (defaut : 10 kg)

Methodes :

Methode UML	Methode Dart / Description
calculerImpact()	communityStatsProvider (StreamProvider) Aggregation automatique Firestore : points top 100 utilisateurs + deletedPoints (impact communautaire preserve apres suppression de comptes)
afficherStatistiques()	StatsScreen - Affiche graphiques CO2, points, tendances, progression hebdomadaire et contributions communautaires
copyWith()	UserModel.copyWith(...) -> UserModel - Mise a jour partielle des statistiques

9. MessageChat -> ChatMessage

Implemente dans : Frontend/lib/models/chat_message.dart

Attributs :

Attribut UML	Type	Attribut Dart / Description
id	String	id - Timestamp en microsecondes (identifiant unique du message)
contenu	String	text - Texte du message (utilisateur ou EcoBot)
type	bool	isUser - true = message utilisateur | false = message EcoBot
date	DateTime	timestamp - Horodatage de creation du message
-	bool	isLoading - true : animation de chargement (bot en train de repondre)
-	bool?	feedback - null = non note | true = pouce haut | false = pouce bas

Methodes :

Methode UML	Methode Dart / Description
envoyerMessage()	factory ChatMessage.user(String text) -> ChatMessage Cree un message utilisateur avec id unique et timestamp automatique
afficherMessage()	factory ChatMessage.bot(String text) -> ChatMessage Cree un message EcoBot avec id unique et timestamp automatique
-	factory ChatMessage.loading() -> ChatMessage Cree un message d'attente avec animation (isLoading = true)
-	static getter ChatMessage.welcome -> ChatMessage Message d'accueil : "Bonjour ! Je suis EcoBot..."
-	copyWith({String? text, bool? isLoading, bool? feedback}) -> ChatMessage Mise a jour partielle d'un message existant

10. Notification -> NotificationService + Firestore

Implemente dans : Frontend/lib/services/notification_service.dart

Attributs (document Firestore : users/{uid}/notifications/{id}) :

Attribut UML	Type	Champ Firestore / Description
id	String	id (doc.id) - Identifiant document Firestore genere automatiquement
titre	String	title - Titre de la notification push ou locale
message	String	body - Corps du message de notification
-	Map	data - Donnees FCM supplementaires (routing, payload)
-	bool	read - false = non lue | true = lue par l'utilisateur
date	Timestamp	createdAt - Timestamp serveur Firestore (FieldValue.serverTimestamp())
-	String	fcmToken - Token FCM de l'appareil (stocke dans users/{uid})

Methodes (toutes static dans NotificationService) :

Methode UML	Methode Dart / Description
envoyerNotification()	_showLocal({String title, String body}) -> Future<void> Affiche une notification locale via FlutterLocalNotificationsPlugin
-	static Future<void> init() Initialise FCM + permissions Android/iOS + timezone Africa/Casablanca + canal ecoscan_channel + ecoute token refresh
-	static Future<void> scheduleDailyReminders() Planifie un rappel quotidien a 8h00 : "Triez vos dechets aujourd'hui et gagnez des points !"
-	static Future<void> scheduleWeeklyChallengeReminder() Planifie un rappel chaque lundi a 8h00 : "Nouveaux defis hebdomadaires !"
-	static Future<void> showNewRewardUnlocked(String rewardName) Affiche une notification "Recompense disponible" quand l'utilisateur a assez de points

-	static Future<void> markAsRead(String notificationId) Met a jour read = true dans Firestore pour la notification donnee
-	static Stream<List<Map>> getNotifications() Flux temps reel des 50 dernieres notifications de l'utilisateur (ordonnees par createdAt descendant)